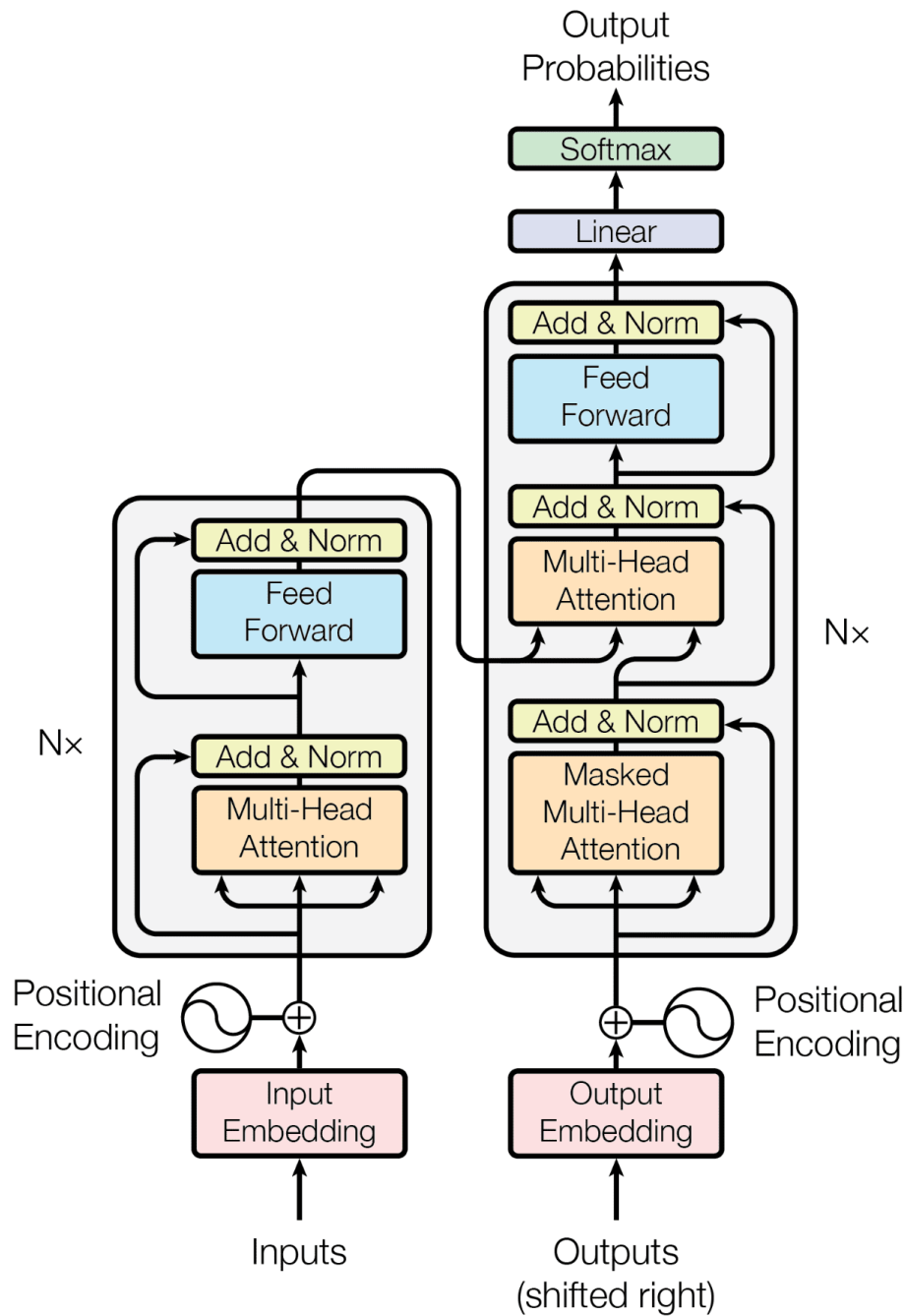


Transformer



```
GPT_CONFIG_124M = {
    "vocab_size": 50257, # total unique tokens in tokenizer vocabulary
    "ctx_length": 1024, # maximum number of input tokens the model can process
    "emb_dim": 768, # size of token embedding / hidden representation
    "n_heads": 12, # number of attention heads in multi-head attention
    "n_layers": 12, # number of transformer blocks (layers)
    "drop_rate": 0.1, # dropout probability used for regularization
    "qkv_bias": False # whether Query/Key/Value projections use bias terms
}
```

DummyGPTModel

```
class DummyGPTModel(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
        self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
        self.drop_emb = nn.Dropout(cfg["drop_rate"])

        # Use a placeholder for TransformerBlock
        self.trf_blocks = nn.Sequential(
            *[DummyTransformerBlock(cfg) for _ in range(cfg["n_layers"])]
        )

        # Use a placeholder for LayerNorm
        self.final_norm = DummyLayerNorm(cfg["emb_dim"])
        self.out_head = nn.Linear(
            cfg["emb_dim"], cfg["vocab_size"], bias=False
        )

    def forward(self, in_idx):
        batch_size, seq_len = in_idx.shape
        tok_embeds = self.tok_emb(in_idx)
        pos_embeds = self.pos_emb(torch.arange(seq_len, device=in_idx.device))
        x = tok_embeds + pos_embeds
        x = self.drop_emb(x)
        x = self.trf_blocks(x)
        x = self.final_norm(x)
        logits = self.out_head(x)
        return logits

class DummyTransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        # A simple placeholder

    def forward(self, x):
        # This block does nothing and just returns its input.
        return x

class DummyLayerNorm(nn.Module):
    def __init__(self, normalized_shape, eps=1e-5):
        super().__init__()
        # The parameters here are just to mimic the LayerNorm interface.

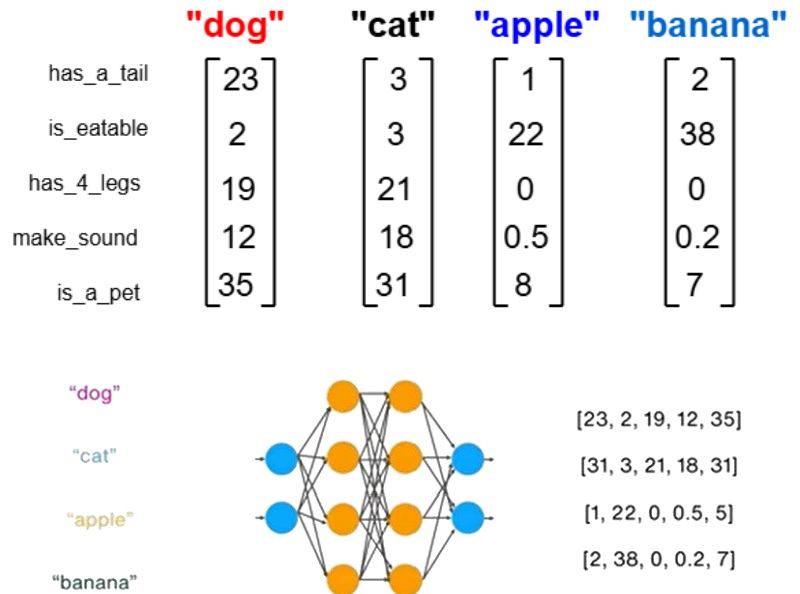
    def forward(self, x):
        # This layer does nothing and just returns its input.
        return x
```

1. Input Embedding

How are these vectors created?

We can train a neural network to turn words into vector embeddings. By looking at sentences in a document, the model learns which words often appear together and which have similar meanings.

But this isn't easy. For example, a model like GPT-2 has a vocabulary of about 50,000 words. That means it needs to learn a vector for each word and understand how they relate to one another. Doing this requires a lot of computation and data



Token String Token ID Embedded Token Vector

'<s>' -> 0 -> [0.1150, -0.1438, 0.0555, ...]
'<pad>' -> 1 -> [0.1149, -0.1438, 0.0547, ...]
'</s>' -> 2 -> [0.0010, -0.0922, 0.1025, ...]
'<unk>' -> 3 -> [0.1149, -0.1439, 0.0548, ...]
'.' -> 4 -> [-0.0651, -0.0622, -0.0002, ...]
' the' -> 5 -> [-0.0340, 0.0068, -0.0844, ...]
' ,' -> 6 -> [0.0483, -0.0214, -0.0927, ...]
' to' -> 7 -> [-0.0439, 0.0201, 0.0189, ...]
' and' -> 8 -> [0.0523, -0.0208, -0.0254, ...]
' of' -> 9 -> [-0.0732, 0.0070, -0.0286, ...]
' a' -> 10 -> [-0.0194, 0.0302, -0.0838, ...]
...

Embedding Model Comparison:

Model	Type	Embedding size (dim)	Vocabulary size	Notes
Word2Vec	Static	300	~3M (Google News)	One vector per word
ELMo	Context-aware (LSTM)	256–1024	No fixed vocab (char-based)	Infinite word handling
DeepSeek	Transformer	~4K–8K	~100K–200K	Efficient scaling
GPT-2 (small)	Transformer	768	50,257	Subword BPE
GPT-3 (Davinci)	Transformer	12,288	~100K	Much larger latent space
Modern LLMs (e.g. GPT-4 class)	Transformer	3K–12K+	100K–200K+	Larger vocab trend

How are these vectors created?

Step1: Start with a tiny dataset:

Imagine we have this small corpus:

"the cat sits on the mat"
"the dog sits on the log"



From this, our vocabulary is:

["the", "cat", "dog", "sits", "on", "mat", "log"]

Step2: Create training pairs (context learning):

One simple method is **Word2Vec (Skip-gram)**.

The idea: given a word, predict its surrounding words.

"the cat sits on the mat"



If we choose a window size of 1, training pairs look like:

- ("the" → "cat"), ("the" → "sits"),
- ("cat" → "the"), ("cat" → "sits"), ("cat" → "on")
- ("sits" → "the"), ("sits" → "cat"), ("sits" → "on")
- (on, cat), (on, sits), (on, the), (on, mat),
- (the, sits), (the, on), (the, mat),
- (mat, on), (mat, the)

So the model learns: *words appearing in similar contexts*

Step3: Convert words to numbers:

We encode each word as a **one-hot vector**:

Example (7 words total):

- "the" → [1,0,0,0,0,0,0]
- "cat" → [0,1,0,0,0,0,0]
- "dog" → [0,0,1,0,0,0,0]

These are sparse and not meaningful yet.

Step 4: Neural network structure:

A very simple network:

Input (one-hot) → Hidden layer → Output layer

- Input size = vocabulary size (7)
- Hidden layer = embedding size (say 3)
- Output = probability of context words

What happens during training

When the model processes:

Case 1: ("cat" → "sits")

- It adjusts weights so **cat's vector** helps predict "sits"

Case 2: ("dog" → "sits")

- It adjusts weights so **dog's vector** helps predict "sits"

Since both:

- need to predict the same outputs
- receive similar gradient updates

Step 5: Training process

For each pair like ("cat" → "sits"):

1. Input: one-hot("cat")
2. Predict: probability distribution over all words
3. Compare with actual ("sits")
4. Compute loss (cross-entropy)
5. Update weights using backpropagation

Repeat this thousands/millions of times.

Step 6: Final result (embeddings)

cat ≈ [0.2, 0.8, 0.1]
dog ≈ [0.25, 0.75, 0.15]
mat = [0.9, 0.1, 0.2]
log = [0.85, 0.15, 0.25]

<pre> # Step 1: Toy dataset sentences = [["the", "cat", "sits", "on", "the", "mat"], ["the", "dog", "sits", "on", "the", "log"]] # Step 2: Build vocabulary import numpy as np words = list(set(w for s in sentences for w in s)) word_to_idx = {w:i for i,w in enumerate(words)} idx_to_word = {i:w for i,w in word_to_idx.items()} vocab_size = len(words) embedding_dim = 2 </pre>	<pre> Step 3: Generate training pairs (window=1) pairs = [] window = 1 for sent in sentences: for i, word in enumerate(sent): for j in range(max(0, i-window), min(len(sent), i+window+1)): if i != j: pairs.append((word, sent[j])) # Step 4: Initialize weights W1 = np.random.randn(vocab_size, embedding_dim) # embeddings W2 = np.random.randn(embedding_dim, vocab_size) # output </pre>
<pre> # Step 5: Helper functions def softmax(x): e_x = np.exp(x - np.max(x)) return e_x / e_x.sum() def one_hot(idx, size): vec = np.zeros(size) vec[idx] = 1 return vec # Step 6: Helper functions lr = 0.05 for epoch in range(5000): loss = 0 for target, context in pairs: x = one_hot(word_to_idx[target], vocab_size) y_true = one_hot(word_to_idx[context], vocab_size) # Forward pass h = np.dot(x, W1) # embedding u = np.dot(h, W2) # scores y_pred = softmax(u) # Loss (cross-entropy) loss -= np.log(y_pred[np.argmax(y_true)] + 1e-9) # Backprop error = y_pred - y_true dW2 = np.outer(h, error) dW1 = np.outer(x, np.dot(W2, error)) # Update W1 -= lr * dW1 W2 -= lr * dW2 if epoch % 1000 == 0: print(f"Epoch {epoch}, Loss: {loss:.4f}") # Step 7: Inspect embeddings for word in words: print(word, W1[word_to_idx[word]]) </pre>	PyTorch Implementation (Skip-gram) <pre> Step 1: Dataset : Same as Step1- Toy dataset Step 2: Training pairs: Same as Step3 - Generate training pairs Step 3: Model # nn.Embedding is just a lookup table (no explicit one-hot needed) class SkipGram(nn.Module): def __init__(self, vocab_size, embedding_dim): super().__init__() self.embeddings = nn.Embedding(vocab_size, embedding_dim) self.output = nn.Linear(embedding_dim, vocab_size) def forward(self, x): x = self.embeddings(x) # this replaces one-hot + matrix multiply x = self.output(x) return x Step 4: Training for epoch in range(1000): total_loss = 0 for target, context in pairs: target = torch.tensor([target]) context = torch.tensor([context]) output = model(target) loss = loss_fn(output, context) optimizer.zero_grad() loss.backward() optimizer.step() total_loss += loss.item() if epoch % 200 == 0: print(f"Epoch {epoch}, Loss: {total_loss:.4f}") Step 5: Training embeddings = model.embeddings.weight.data for word, idx in word_to_idx.items(): print(word, embeddings[idx]) </pre>

2. Positional Embedding

1. The core problem:

Attention has no sense of order

1. "I deposited money in the bank"
2. "bank the in money deposited I"

Or,

1. "dog bites man"
2. "man bites dog"

Even though both contain the same words, But meaning depends heavily on order.

Token embeddings for both might look identical as a set:

["dog", "bites", "man"]
["man", "bites", "dog"]

If the model only uses embeddings:

- it sees same tokens
- just shuffled

So it cannot distinguish meaning.
But meaning changes completely.

2. Why position matters (your example: "bank")

Self-attention (core of Transformers) works like this:

- compares every token with every other token
- uses similarity (dot product)
- does NOT inherently know position

So without extra information:

"cat chased mouse"
"mouse chased cat"

would look structurally similar.

Consider the sentence:

"I deposited money in the bank"

Here, "bank" = financial institution

But compare:

"I sat on the river bank"

Here, "bank" = river edge.

4. Types of positional encoding

4.1. Fixed (Sinusoidal – original Transformer)

Even dimensions:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

Odd dimensions:

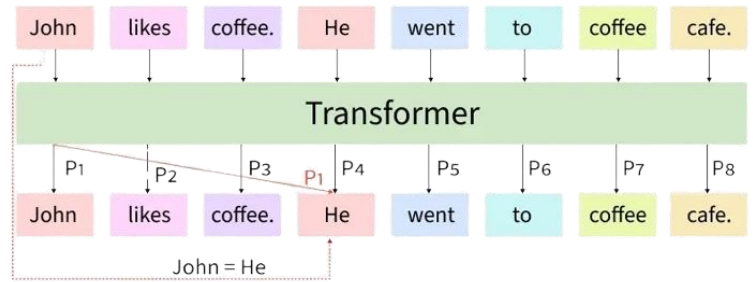
$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- pos = word position in sentence
- d = embedding dimension (e.g., 512 or 768)
- i = dimension index

<https://huggingface.co/blog/designing-positional-encoding>

<https://blog.timodenk.com/linear-relationships-in-the-transformers-positional-encoding/>

<https://towardsdatascience.com/master-positional-encoding-part-i-63c05d90a0c3/>



3. Why self-attention alone is not enough

If we remove positional encoding, the model sees a bag of words:

I, deposited, money, in, the, bank

Self-attention computes relationships like:

"Which words are relevant to each other?"

But without positional encoding, it does NOT know:

"What is the order of those words?"

So "money → bank" and "bank → money" would look identical in a bag-of-words sense.

Note: Why not just use numbers like 1, 2, 3, 4...?

We could assign:

- "I" → 1, "deposited" → 2, "money" → 3

This causes problems:

- The model might think position 10 is "10× bigger" than position 1 (which is not meaningful in language)
- It doesn't generalize well to longer sequences than it saw during training

So instead, Transformers use a smooth mathematical pattern: **sine and cosine waves**.

position	sin(pos)	cos(pos)
1	0.84	0.54
2	0.91	-0.42
3	0.14	-0.99

"bank at position 6" ≠ bank at position 2"
even if the word is identical

4.2. Learned positional embeddings (used in GPT-style models)

Instead of fixed formulas:

- each position has a learnable vector
- model learns best representation during training

4.3. Relative positional encoding (modern models)

Instead of absolute position, it encodes:

- "how far apart two tokens are"

This helps:

- better long-context understanding
- better generalization

Why Alternate Sin and Cos?

Idea 1: Just Count (Integer Indexing)

- **The Idea:** The first word is 1, the second is 2, the third is 3, and so on. [1, 2, 3, 4, ...]
- **The Flaw:** The numbers can become very large for long sentences. Neural networks work best with small, normalized values, and large numbers can make training unstable. Also, a model might see very different ranges of numbers during training vs. testing, making it hard to generalize.

Idea 2: Normalize the Count

To fix the scaling issue, the next logical step is to normalize the indices to a fixed range, like [0, 1].

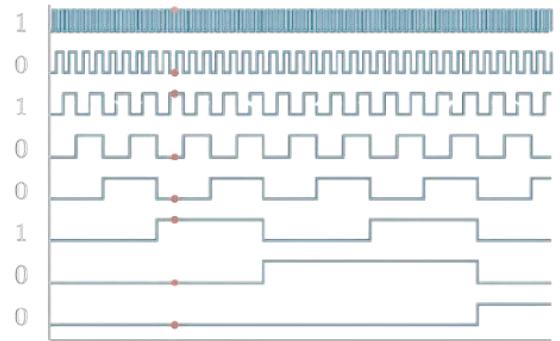
- **The Idea:** Divide each index by the length of the sentence. For a 4-word sentence, the encoding would be: [1/4, 2/4, 3/4, 4/4] or [0.25, 0.5, 0.75, 1.0]
- **The Flaw:** This makes the meaning of a position dependent on the sentence length. For example, the value 0.5 means the 2nd position in a 4-word sentence, but it would mean the 10th position in a 20-word sentence. The model has no consistent way to interpret what a position value means, which is a deal-breaker.

so we need static information which represents a position it cant be dependent on the length of the sentence or some other variable factor.

Idea 3: Use a Binary Vector

To handle arbitrary lengths consistently while keeping values small, the next idea is to represent the position index in binary and turn it into a vector.

- **The Idea:** Represent each number with its binary equivalent, creating a vector for each position. Here we solved the previous two problems which were keeping the values between 0 and 1 and keeping the information static regardless of the length of sentence (still all the numbers are positive though).
 - Position 2 -> [0, 0, 1, 0]
 - Position 3 -> [0, 0, 1, 1]
 - Position 4 -> [0, 1, 0, 0]
- **The Flaw:** This is discrete and "jagged." A small change in position (e.g., from 7 (0111) to 8 (1000)) can cause every single value in the vector to flip. This gives the model no smooth, continuous sense of distance or proximity between nearby positions.



Idea 4: Using Sine Waves

Alright so we need something which is smooth, meaning it periodically changes from 0 to 1 to 0 and so on, well sine function can perform this perfectly and its also in the range of [-1,1], which solves our problem of having equal positive and negatives.

1. Periodicity problem

- Sine is **periodic**, which means it repeats its values over and over.
- For example, if we move along the sine wave by 1 radian per position:
 - $\sin(1), \sin(2), \dots$
 - By the time we reach $\sin(6)$, the value is already **very close to what we had earlier**, just because sine repeats.
- This creates a problem: the model sees **almost the same number at different positions**, even though they are far apart in the sequence. It can't reliably tell positions apart.

2. Phase problem

- Even without worrying about repetition, **phase ambiguity** is an issue.
- Consider $\sin(30)$ and $\sin(150)$. Both return roughly 0.5.
- For the model, if you only provide the sine value, it sees **0.5** and has **no idea whether it came from position 30 or 150**.
- This is why **sine alone doesn't uniquely identify positions** — the same value can appear at many different points in the sequence.
- You cant locate a point in a 2d space just with a y axis (sine)

3. Linear Transformation

Another major problem is **linear transformation**. You might think that moving from pos = 6 to pos = 9 should be equivalent to moving from pos = 18 to pos = 21, because the **offset k = 3 is the same**. But it won't happen with the current approach

But before we look at why it won't happen, let's understand why we want this property? This is very well explained in a blog that I'm quoting here

Why is this a very desirable property to have? Imagine we have a network that is trying to translate the sentence "I am going to eat."

The combination "is/am/are" + "going" + "to" + "verb" is a very common grammatical structure, with a fixed positional structure. "going" always ends up at index 1, "to" at index 2, etc.

In this case, when translating "verb", we may want the network to learn to pay attention to the noun "I" that occurs before "am going to". "I" is located 4 position units to the left of "verb".

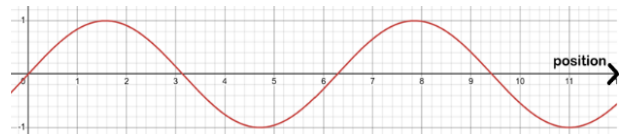
Since our attention layer uses linear transformations to form the keys, queries, and values, it would be nice if we have positional encodings such that the linear transformation can translate the position vector located 4 units to the left, so that it lines up with the position vector of "verb".

The query and key would then match up perfectly.

Mathematical Reason:

- Bounded values $[-1, 1]$

	"dog"	bites	man"
pos->	1	2	3



Approach1: Single Sin function:

If we use a simple sin function

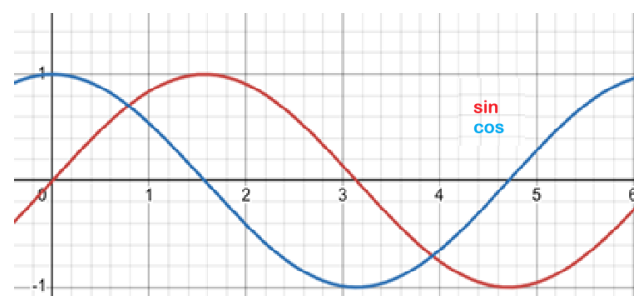
- then for position 1 ("dog") $\Rightarrow \sin(1) = 0.84$
- then for position 2 ("bites") $\Rightarrow \sin(2) = 0.87$
- then for position 3 ("man") $\Rightarrow \sin(3) = 0.19$
- Final positional embedding for "dog bites man" $\Rightarrow [0.84, 0.87, 0.19]$

Problem: As sin function is periodic (i.e., pattern is repetitive) and it can produce same value for other position also. Here in above example for the sentence "dog bites man", as per this sin function the position of "dog" = pos(1) = $\sin(1) = 0.84$, but as it's a periodic function it might happen some other word/token also calculate the same position, so this value is not unique. i.e. different word in same sentence may have same positional encoding value.

Approach2: Combination of Sin & cosine:

What if we use 2 separate functions, one is $\sin(\text{pos})$ and other is $\cos(\text{pos})$. i.e., for a single word there would be 2 values one will come from sin and another from cos

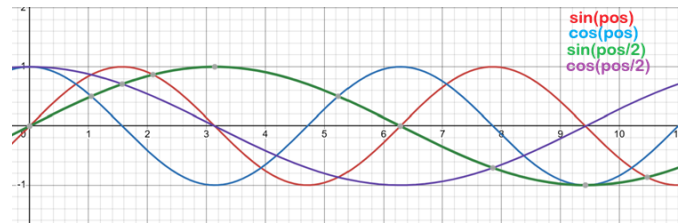
- then for position 1 $\Rightarrow \sin(1) = 0.84, \cos(1) = 0.5$
- then for position 2 $\Rightarrow \sin(2) = 0.87, \cos(2) = -0.4$
- then for position 3 $\Rightarrow \sin(3) = 0.19, \cos(3) = -1$
- Final positional embedding for "dog bites man" $\Rightarrow [[0.84, 0.5], [0.87, -0.4], [0.19, -1]]$



Problem: still pattern may be repeat (duplicate positional embedding)

Approach3: Combination of multiple Sin & cosine:

- $\sin(\text{pos})$, $\cos(\text{pos})$, $\sin(\text{pos}/2)$, $\cos(\text{pos}/2)$
- i.e., we will use 4 combination to represent the position of a single word
- i.e, we can reduce the probability to have the duplicate position



But how much maximum? Any limit?

- $\sin(\text{pos})$, $\cos(\text{pos})$, $\sin(\text{pos}/2)$, $\cos(\text{pos}/2)$, $\sin(\text{pos}/3)$, $\cos(\text{pos}/3)$,

Why Alternate Sin and Cos?

Using both sine and cosine allows unique representation of position and offset.

Because: $\sin(a + b) = \sin(a)\cos(b) + \cos(a)\sin(b)$

It means:: relative position information becomes computable from combinations.

Why Divide by 10000?

This creates multiple frequencies across dimensions.

Without it: all dimensions would oscillate identically.

Why Different Frequencies? $10000^{2i/d}$

⇒ controls wavelength/frequency.

Small Dimensions

- For small i : denominator small.
- So angle changes rapidly.

Large Dimensions

- For large i : denominator huge.
- Angle changes slowly.

Example:

Suppose: $d = 4$,

dim	Formula
0	$\sin(\text{pos})$
1	$\cos(\text{pos})$
2	$\sin(\text{pos}/100)$
3	$\cos(\text{pos}/100)$

Because:

- for $i = 0$: $10000^{0/4} = 1$
- for $i = 1$: $10000^{2/4} =$

	<-----Embedding Dim----->			
words	$\sin(0/n^{2*0/512})$	$\cos(0/n^{2*0/512})$		$\cos(0/n^{2*256/512})$
	$\sin(1/n^{2*0/512})$	$\cos(1/n^{2*0/512})$		$\cos(1/n^{2*256/512})$
	$\sin(2/n^{2*0/512})$	$\cos(2/n^{2*0/512})$		$\cos(2/n^{2*256/512})$
	$\sin(3/n^{2*0/512})$	$\cos(3/n^{2*0/512})$		$\cos(3/n^{2*256/512})$
				
	$\sin(M\text{pos}/n^{2*0/512})$	$\cos(M\text{pos}/n^{2*0/512})$		$\cos(M\text{pos}/n^{2*256/512})$

pos	Dim 0 ($\sin(\text{pos})$)	Dim 1 ($\cos(\text{pos})$)	Dim 2 ($\sin(\text{pos}/100)$)	Dim 3 ($\cos(\text{pos}/100)$)	Final Positional Encoding
0	$\sin(0)=0$	$\cos(0)=1$	$\sin(0)=0$	$\cos(0)=1$	[0, 1, 0, 1]
1	$\sin(1)=0.841$	$\cos(1)=0.540$	$\sin(0.01)=0.01$	$\cos(0.01)=0.99995$	[0.841, 0.540, 0.01, 0.99995]
2	$\sin(2)=0.909$	$\cos(2)=-0.416$	$\sin(0.02)=0.02$	$\cos(0.02)=0.9998$	[0.909, -0.416, 0.02, 0.9998]
3	$\sin(3)=0.141$	$\cos(3)=-0.989$	$\sin(0.03)=0.03$	$\cos(0.03)=0.9995$	[0.141, -0.989, 0.03, 0.9995]

Observation 1 — Fast Oscillation

Dimensions 0 and 1:

pos	$\sin(\text{pos})$	$\cos(\text{pos})$
0	0	1
1	0.841	0.540
2	0.909	-0.416
3	0.141	-0.989

These change rapidly.

- local order
- nearby token relationships

Observation 2 — Fast Oscillation

Dimensions 2 and 3:

pos	$\sin(\text{pos}/100)$	$\cos(\text{pos}/100)$
0	0	1
1	0.01	0.99995
2	0.02	0.9998
3	0.03	0.9995

These change slowly.

- broader sequence structure
- long-range positioning

Let's walk through a transformer mini example:

"The cat sat on the mat"

and understand:

1. How "cat" at position 1 differs from "cat" at position 3
2. What happens after adding positional encoding
3. How attention uses this
4. How the model finally understands: "cat sat on mat"

1. Step 1 — Word embeddings

Embedding("cat") = [2, 1, 0, 3]

Embedding("sat") = [1, 3, 2, 0]

Embedding("mat") = [0, 2, 4, 1]

2. Step 2 — Positional encodings

Suppose:

- Position 1: $PE(1) = [0.84, 0.54, 0.01, 0.99]$
- Position 3: $PE(3) = [0.14, -0.99, 0.03, 0.95]$

These are generated from sine/cosine waves.

3. Step 3 — Add them

"cat" at position 1 and "cat" at position 3:

$cat@1 = [2, 1, 0, 3] + [0.84, 0.54, 0.01, 0.99]$
 $= [2.84, 1.54, 0.01, 3.99]$

$cat@3 = [2, 1, 0, 3] + [0.14, -0.99, 0.03, 0.95]$
 $= [2.14, 0.01, 0.03, 3.95]$

4. Important observation

large first dimension

large fourth dimension

But positions slightly shift them: $cat@1 \neq cat@3$

5. How does transformer detect position?

Attention compares vectors using dot products.

→ $score = Q \cdot K$

If vectors are similar → large score.

6. Sentence example

Positions:

The → 0, cat → 1, sat → 2

on → 3, the → 4, mat → 5

Suppose model is processing "sat".

It asks:

"Who sat?"

So "sat" attends strongly to nearby noun "cat".

7. Why does position matter?

Without positional encoding:

cat sat mat

mat sat cat

would produce nearly same attention relationships.

Transformer would not know word order.

With positional encoding:

$cat@1$ and $cat@5$ are different vectors.

So attention scores change.

8. Real intuition using vector differences

$$(E + P_3) - (E + P_1) = P_3 - P_1$$

Compute it:

$cat@3 - cat@1 =$
 $[2.14, 0.01, 0.03, 3.95] - [2.84, 1.54, 0.01, 3.99]$
 $=$
 $[-0.70, -1.53, 0.02, -0.04]$

This difference equals positional shift.

The "cat" meaning cancels out.

So the model learns:

"This directional change means token moved right."

9. How model understands "cat sat"

Suppose "sat" vector attends to nearby words.

Attention scores might become:

Word	Attention from "sat"
The	0.1
cat	0.9
on	0.3
mat	0.2

Highest attention: **sat → cat**

So model learns: **cat is subject of sat**

Position helps because:

- "cat" is BEFORE "sat"
- nearby tokens often relate grammatically

10. How does it know "mat" is object/location?

Now "sat" also attends to "mat":

cat sat on mat

Pattern learned during training:

noun + sat + on + noun

usually means:

subject sat on object

11. Why addition works mathematically

Neural layers are linear transformations.

A linear layer computes: Wx

Now suppose: $x = e + p$ $e = \text{embedding}$

Then: $W(e + p)$ $p = \text{positional encoding}$

Using linearity: $We + Wp$

Eg: $\sin(x + \Delta)$ can be represented as $\sin(x)\cos(\Delta) + \cos(x)\sin(\Delta)$ using linear relationships.

$$\sin(a + b) = \sin(a)\cos(b) + \cos(a)\sin(b)$$

$$\text{and } \cos(a + b) = \cos(a)\cos(b) - \sin(a)\sin(b)$$

→ Position $k + \Delta$ can be computed from position k .

Key insight:

Transformer NEVER explicitly stores: position = 3, instead it learns geometric relationships.

Like:

vector shifted this way = later token

vector shifted opposite way = earlier token

Position becomes geometry.

<pre> # Word embeddings X = np.array([[1, 0, 1], # I [0, 2, 1], # love [1, 1, 0] # cats]) # Absolute positional embeddings P = np.array([[0.1, 0.1, 0.1], # position 0 [0.2, 0.2, 0.2], # position 1 [0.3, 0.3, 0.3] # position 2]) # Final embeddings X_final = X + P </pre>	<pre> vocab_size = 50257 output_dim = 768 embeddings = torch.nn.Embedding(vocab_size, output_dim) ctx_length = 4 # lets sat we want to process 4 tokens at a time pos_embedding_layer = nn. Embedding(ctx_length, output_dim) pos_embeddings = pos_embedding_layer(torch.arange(ctx_length)) print(pos_embeddings.shape) # 4x256 input_embeddings = token_embeddings + pos_embeddings </pre>
---	--

The Attention Mechanism (Self Attention without Trainable weights)

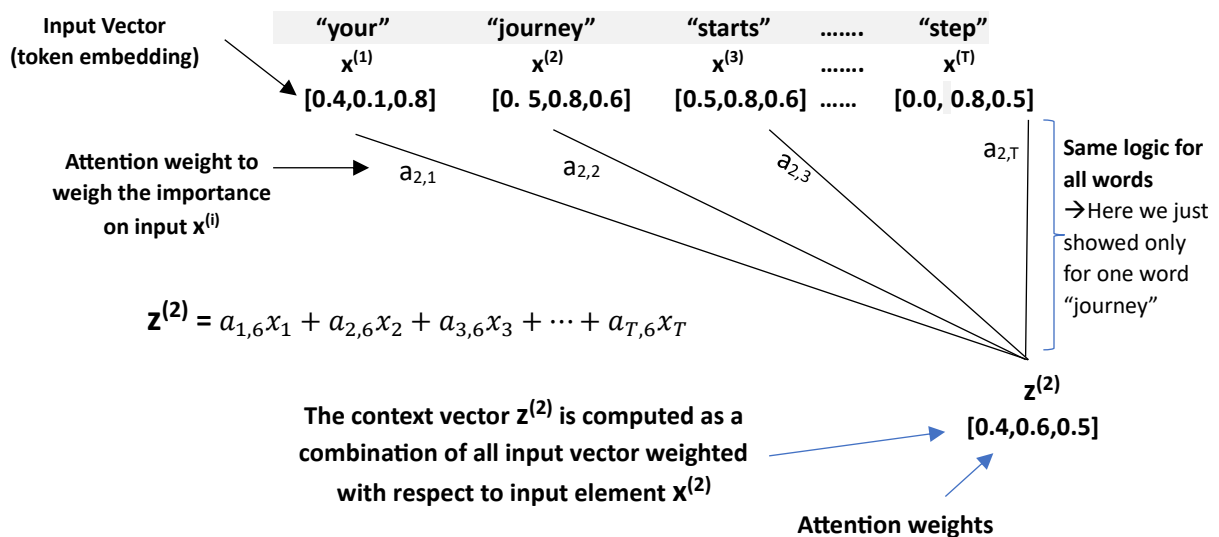
Context matters

Word	Example Context
see	I will see you soon I will see this project to its end I see what you mean
bank	I went to the bank to apply for a loan I am banking on the job offer coming through I am standing on the left bank
it	The animal didn't cross the street because it was too tired The animal didn't cross the street because it was too wide
station	The train left the station on the time The radio station was playing 60s hit I was stationed on a remote island in Polynesia

Self Attention
with trainable weights

Causal Attention
(only consider
previous & current
input in a sequence)

Multi-Head Attention
(only consider
previous & current
input in a sequence)



How should we achieve this?

- Dot Product Similarity:

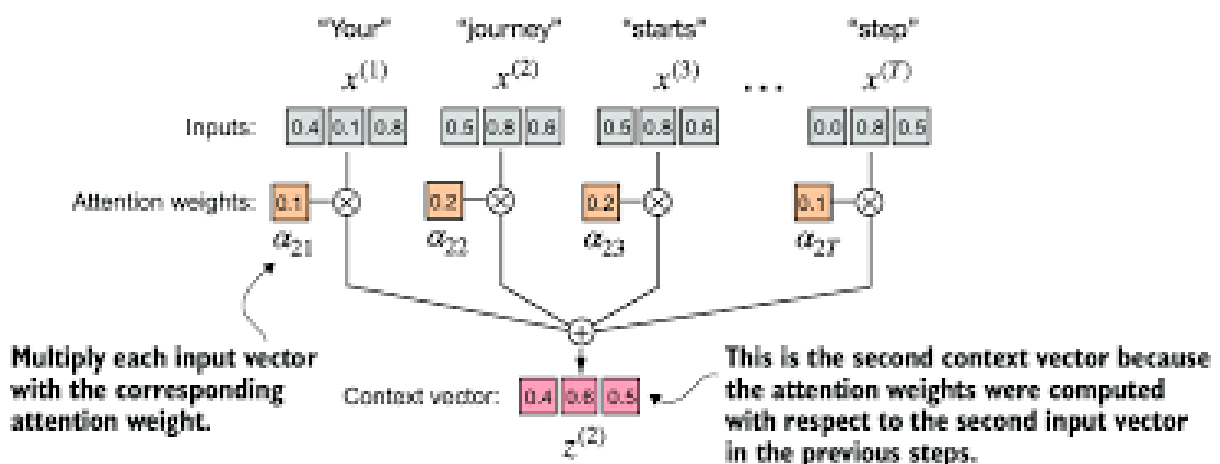
$$\langle \text{journey}, \text{step} \rangle = \|\text{journey}\| \|\text{step}\| \cos(\theta)$$

- However, since $\cos(\theta)$ can be negative, these similarity scores cannot be used directly as weights. We need weights that are:
 - Non-negative
 - Sum to 1

Solution: Softmax Normalization

$$s_{1,6} = \frac{\exp(\langle x_1, x_6 \rangle)}{\sum_k \exp(\langle x_k, x_6 \rangle)}$$

	Your	journey	starts	with	one	step
Your	0.20	0.20	0.19	0.12	0.12	0.14
journey	0.13	0.23	0.23	0.12	0.10	0.15
starts	0.13	0.23	0.23	0.12	0.11	0.15
with	0.14	0.20	0.20	0.14	0.12	0.17
one	0.15	0.19	0.19	0.13	0.18	0.12
step	0.13	0.21	0.21	0.14	0.09	0.18



Implement Simple Attention mechanism

#Step1: Convert Words into Vectors <pre>inputs = torch.tensor([[0.43, 0.15, 0.89], # your (x^1) [0.55, 0.87, 0.66], # journey (x^2) [0.57, 0.85, 0.64], # starts (x^3) [0.22, 0.58, 0.33], # with (x^4) [0.77, 0.25, 0.10], # one (x^5) [0.05, 0.80, 0.55] # step (x^6)])</pre>	#Step2: Compute Similarity # Lets take the 2nd token "journey" # The attention scores are calculated between the # query token and each input tokens <pre>query = inputs[1] # 2nd input token is the query attn_scores_2 = torch.empty(inputs.shape[0]) for i, x_i in enumerate(inputs): attn_scores_2[i] = torch.dot(x_i, query) print(attn_scores_2) # [0.9, 1.4, 1.4, 0.8, 0.7, 1.0]</pre>
#Step3: Softmax Normalization # Normalize each of the attention scores # to obtain attention weights that sum up to 1 <pre>attn_weights_2 = torch.softmax(attn_scores_2, dim=0) print("Attention Weights: ", attn_weights_2) print("Sum: ", attn_weights_2.sum()) #1</pre>	<pre># [0.1385, 0.2379, 0.2333, 0.1240, 0.1082, 0.1581] your journey starts with one step 13% 23% 23% 12% 10% 15% i.e., High attention is paid to journey(23%) and starts(23%)</pre>
# Step4: Weighted Sum (Context Vector) # Calculate Context Z(2) Vector as # weighted sum of all input vectors # This involves multiplying each input vector by its corresponding attention weight <pre>0.1385[0.43, 0.15, 0.89] + 0.2379[0.55, 0.87, 0.66] + 0.2333[0.57, 0.85, 0.64] + 0.1240[0.22, 0.58, 0.33] + 0.1082[0.77, 0.25, 0.10] + 0.1581[0.05, 0.80, 0.55]</pre> <pre>query = inputs[1] # 2nd input token is the query context_vec_2 = torch.zeros(query.shape) for i, x_i in enumerate(inputs): context_vec_2 += attn_weights_2[i] * x_i print(context_vec_2) # [0.4419, 0.6515, 0.5683]</pre>	# Step5: Now we can extend this computation to calculate attention weights and context vectors for all inputs First, we add an additional for-loop to compute the dot products for all pairs of inputs <pre>attn_scores = torch.empty(6,6) for i, x_i in enumerate(inputs): for j, x_j in enumerate(inputs): attn_scores[i,j] = torch.dot(x_i, x_j) print(attn_scores)</pre> ↓ # Optimization: Avoid nested for loop attn_scores = inputs @ inputs.T print(attn_scores)
# Step6: Normalize each row, so that the values in each row sums to 1 <pre>attn_weights = torch.softmax(attn_scores, dim=-1) print(attn_weights)</pre>	# Step7: we now use these attention weights to compute all context vectors <pre>all_context_vecs = attn_weights @ inputs print(all_context_vecs)</pre>

Why do Transformers need trainable weight (Q,K,V) matrices in self-attention? Why can't we directly compute attention using only raw embedding dot products?

Problem With This Simple Method (Dot Product)

"The bank was flooded after heavy rain."
"The cat sat on the mat because it was warm"

If we use raw embedding similarity:

- "bank" may strongly match "money", because embedding contains financial meaning
- **Without Trainable weights:** if we only use dot product between the query "warm" with each word's embedding, we might find that "warm" is most similar to itself, and may be somewhat related to "mat" (if our embeddings capture that mats can be warm). Words like "the", "cat" and "sat" might have low similarity scores because they are not semantically related to "warm"

But in this sentence:

- river-bank meaning is correct

Self attention with learned weights

Intuition

- **Query:** What am I searching for?
- **Key:** What information do I contain?
- **Value:** What information should I send?

Example:

"The bank approved the loan because it trusted the customer"

For word "it":

- Query asks: "Who can trust someone?"
- Keys from other words respond:
 - bank → strong match
 - loan → weak match
 - customer → medium

Thus "it" attends strongly to "bank".

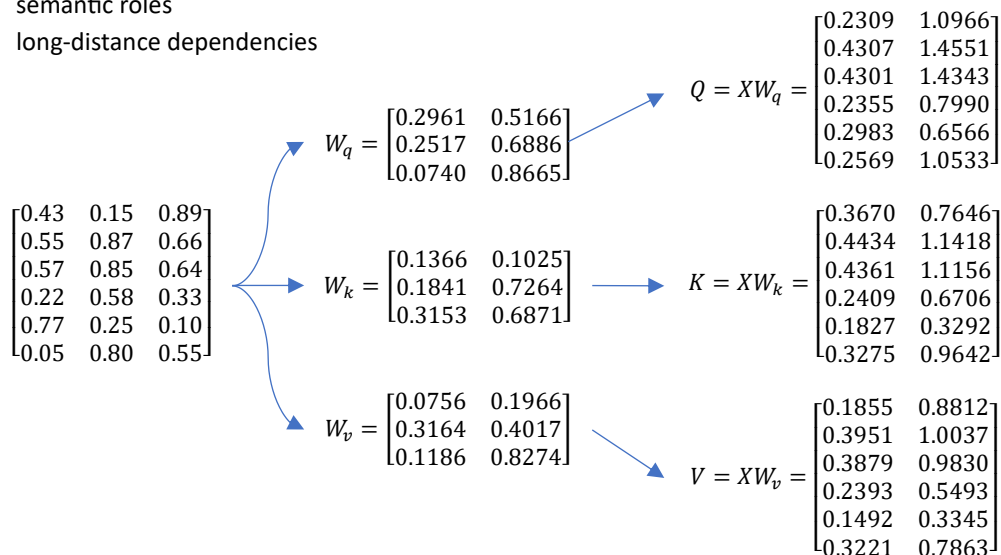
Why Weights Matter?

Without weights:

- similarity only depends on raw embeddings

With learned weights model learns:

- subject-object relations
- tense
- semantic roles
- long-distance dependencies



Full Transformer Formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Why divide by $\sqrt{d_k}$?

- Dot products become very large in high dimensions.
- Large values make softmax unstable.
- Scaling keeps gradients stable.

Example2

"Jaguar runs fast"

Problem:

- Jaguar = animal?
- Jaguar = car?

Without Trainable Weights

Suppose embeddings:

Word	Vector
Jaguar	[5,5]
runs	[1,1]
fast	[2,2]

Dot products:

$$\text{Jaguar} \cdot \text{fast} = 20$$

But this similarity is vague.

Embedding contains BOTH:

- animal meaning
- car meaning

No specialization.

With Trainable Weights:

$$\text{Jaguar} = [\text{animal} = 5, \text{vehicle} = 5]$$

$$W_Q = \begin{bmatrix} 2 & 0 \\ 0 & 0.2 \end{bmatrix} \Rightarrow \text{Transformed: } [5, 5] \begin{bmatrix} 2 & 0 \\ 0 & 0.2 \end{bmatrix} = [10, 1]$$

Now:

- animal feature dominates
- vehicle feature suppressed

The model learned:

in this context, "Jaguar" behaves more like an animal.

Another Important Reason

Different Relationships Need Different Spaces

Relationship	Example
subject-verb	dog runs
pronoun reference	John lost his bag
tense	ate vs eating
location	lives in Paris
sentiment	great movie

Step 1 — Initial Embeddings

Word	Initial Meaning
John	person
dog	animal
hungry	biological state

Initially these are just vectors. → The model does NOT yet know grammar.

Step 2 — Self-Attention Comparison

When processing "it"

→ Transformer computes attention scores with all words.

Compared Word	Attention Score
John	medium
dog	HIGH
hungry	low

Example 1 — Pronoun Resolution

"John gave food to the dog because it was hungry."

Question: Who is "it"?

- John?
- dog?
- food?

Humans know:

- dogs become hungry
- food is not hungry

Transformer learns this statistically.

Why does "dog" get high score?

Because during training, model repeatedly saw patterns like:

"dog was hungry"

"cat was hungry"

"child was hungry"

So weights evolve to associate:

- hungry ↔ living things

Example 2 — Grammar Learning

"The cats are sleeping"

During training the model repeatedly sees:

- cats are
- dog is
- boys are
- girl is

Eventually attention heads learn agreement patterns.

- **One Attention Head May Learn: plural noun ↔ plural verb**
- **Another Head May Learn: subject ↔ action**

Example Attention Pattern

"The cats are sleeping"

The word "eats" may strongly attend to:

- cat

because verbs depend on subjects.

How This Emerges Mathematically

- Suppose: Q_{eats} learns to ask:
→ "Who is performing this action?"
- And keys: K_{cat} learn to advertise:
→ "I am a noun that can perform actions."

Training Process Behind It

Transformer training objective:

→ Predict missing/next words.

Example:

"The dogs ___ barking"

Correct answer: **are**, Wrong: **is**

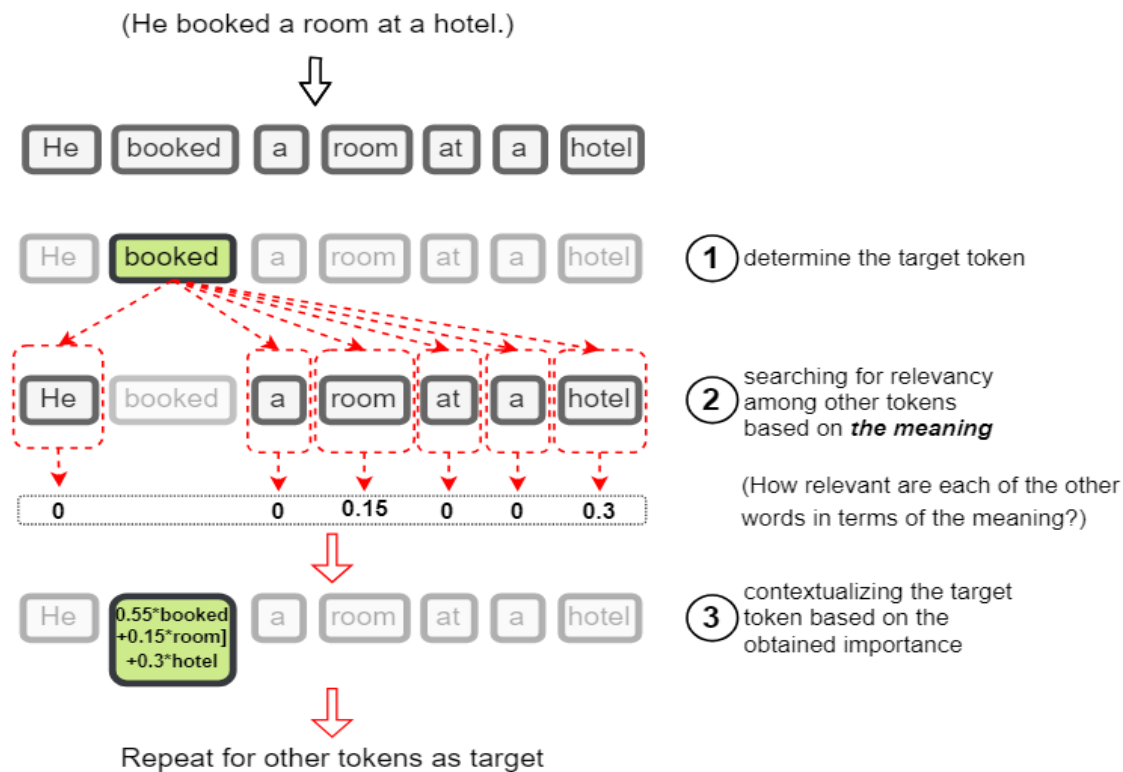
If model predicts wrong:

- gradients update weights
- attention patterns improve

Over billions of examples:

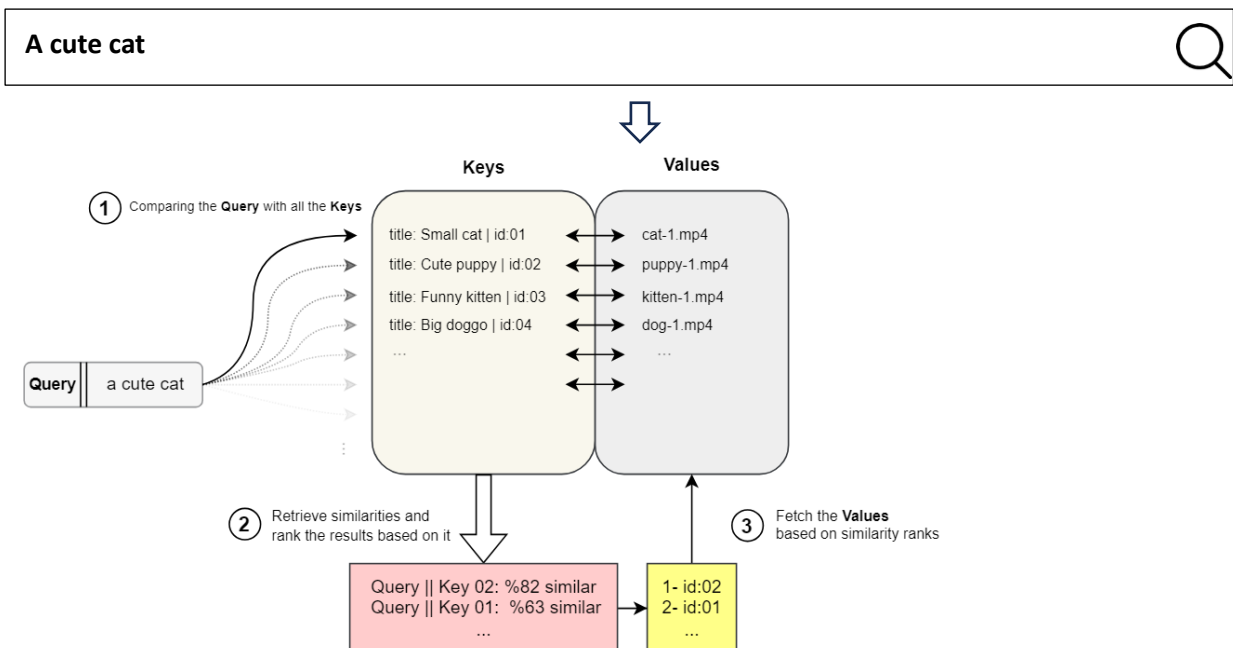
grammar structures emerge naturally.

What is QKV in Transformer Architecture



To make more sense, think of when you search for something on YouTube, for example. Assume YouTube stores all its videos as a pair of "video title" and the "video file" itself. Which we call a Key-Value pair, with the Key being the video title and the Value being the video itself.

The text you put in the search box is called a Query in search terms. So in a sense, when you search for something, YouTube compares your search Query with the Keys of all its videos, then measures the similarity between them, and ranks their Values from the highest similarity down.

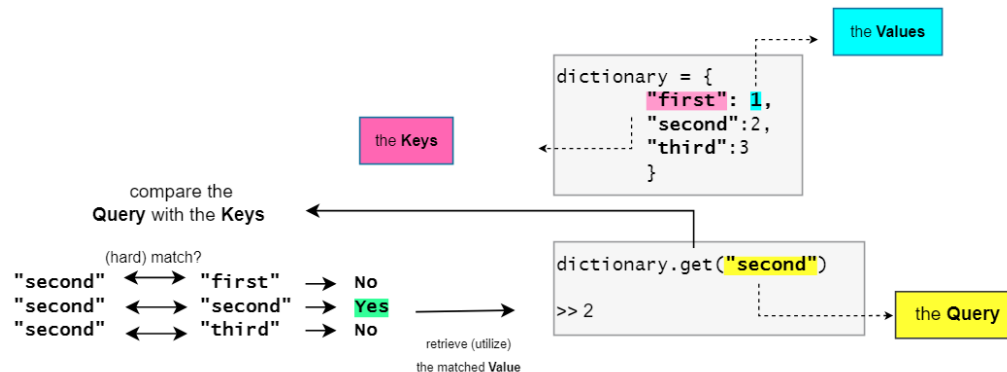


In our problem, we have a sequence of token vectors, and we want to search for the weights to re-weight and contextualize each token (word) embedding of the sequence, we can think in terms of:

1. What you want to look for is the Query.
2. What you are searching among is Key-Value pairs.
3. The query is compared to all the Keys to measure the relevance/importance/similarity.
4. The Values are utilized based on the assigned similarity measure.

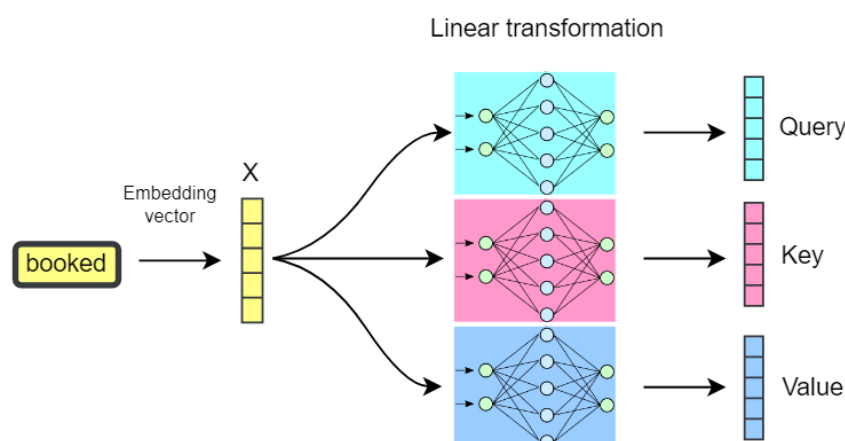
Another helpful relevant analogy is a dictionary (or hashmap) data structure. A dictionary stores data in key-value pairs and it maps keys to their respective value pairs. When you try to get a specific value from the dictionary, you have to provide a query to match its corresponding key, then it searches among those keys, compares them with the query, and if matched, the desired value will be returned.

However, the difference here is that this is a “hard-matching” case, where the Query either exactly matches the Key or it doesn’t and an in-between similarity is not measured between them.



We earlier mentioned that we are only working with real-valued vectors (token embeddings). So the Query, Key, and Value also need to be vectors. However, so far we only have one vector for each token which is its embedding vector. So, how should we obtain the Query, Key, and Value vectors?

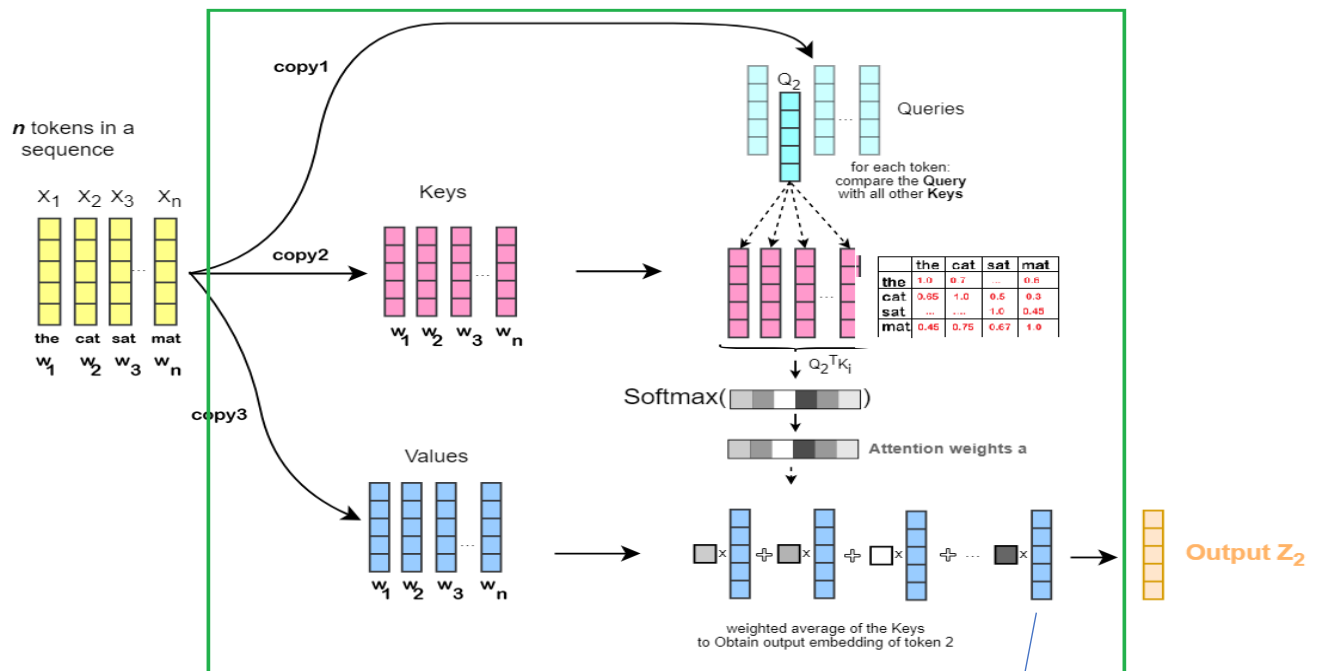
We Construct them using linear projections (linear transformations aka single dense layer with separate sets of weights: W_q , W_k , W_v) of the embedding vector of each token. This means we use a learnable vector of weights for each of the Query, Key, and Value to do a linear transformation on the word embedding to obtain the corresponding Query, Key, and Value vectors.



An embedding of a token may represent different contextual, structural, and syntactical aspects of that token. By using learnable linear transformation layers to construct these vectors from the token’s embedding, we allow the network to:

network to:

1. Extract and pass a limited specific part of that information into the Q , K , and V vectors.
2. Determine a narrower context in which the search and match is going to be done.
3. Learn what information in an embedding is more important to attend to.

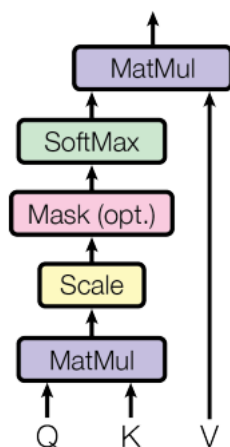


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

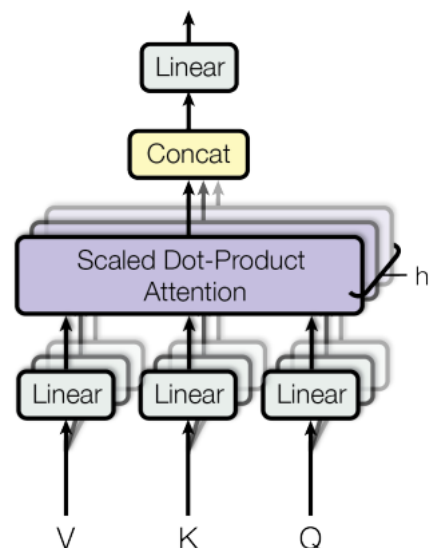
the	$0.1w_1 + 0.7w_2 + 0.2w_3 + \dots$	$\hat{w}_1$
cat	$0.1w_1 + 0.7w_2 + 0.2w_3 + \dots$	$\hat{w}_2$
sat	$0.1w_1 + 0.7w_2 + 0.2w_3 + \dots$	$\hat{w}_3$
mat	$0.1w_1 + 0.7w_2 + 0.2w_3 + \dots$	$\hat{w}_n$

So the whole notion of the Q , K , and V vectors is like a soft dictionary to mimic a *search-and-match procedure* from which we learn how much two tokens in a sequence are relevant (the weights), and what should be added as the context (the values). Also, note that this process does not have to happen sequentially (one token at a time). This all happens in parallel by using matrix operations.

Scaled Dot-Product Attention



Multi-Head Attention



Contextual Embedding in Transformer

Input sentence:

"the cat sits on the mat"

Step 1: Tokenization

["the", "cat", "sits"] → [5, 12, 33]

Step 2: Embedding lookup

```
[5, 12, 33] →  
[  
  [0.1, 0.8, ...], # "the"  
  [0.2, 0.7, ...], # "cat"  
  [0.3, 0.6, ...]  # "sits"  
]
```

👉 Same idea as your model.

What's different in transformers?

Word2Vec:

- Static embeddings (same vector always)

Transformers:

- **Contextual embeddings**

Example:

"bank" in:
- "river bank"
- "bank account"

Different vectors!

How transformers achieve Self-attention?

Instead of:

word → fixed vector

They do:

word → embedding → attention → context-aware vector



Token IDs



Embedding Layer ← (what you implemented)



+ Positional Encoding



Transformer Blocks (**attention**)



Output probabilities

Why positional encoding? (why order matter?)

1. The core problem:

Attention has no sense of order

1. "I deposited money in the bank"
2. "bank the in money deposited I"

Even though both contain the same words, But meaning depends heavily on order

2. Why position matters (your example: "bank")

Consider the sentence:

"I deposited money in the bank"

Here, "bank" = financial institution

But compare:

"I sat on the river bank"

Here, "bank" = river edge.

The word "bank" is ambiguous. The meaning is resolved by context and position:

Even dimensions:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

Odd dimensions:

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- pos = word position in sentence
- d = embedding dimension (e.g., 512 or 768)
- i = dimension index

3. Why self-attention alone is not enough

If we remove positional encoding, the model sees a bag of words:

I, deposited, money, in, the, bank

Self-attention computes relationships like:

"Which words are relevant to each other?"

But without positional encoding, it does NOT know:

"What is the order of those words?"

So "money → bank" and "bank → money" would look identical in a bag-of-words sense.

Note: Why not just use numbers like 1, 2, 3, 4...?

We *could* assign:

- "I" → 1, "deposited" → 2, "money" → 3

This causes problems:

- The model might think position 10 is "10× bigger" than position 1 (which is not meaningful in language)
- It doesn't generalize well to longer sequences than it saw during training

So instead, Transformers use a smooth mathematical pattern: **sine and cosine waves**.

position	sin(pos)	cos(pos)
1	0.84	0.54
2	0.91	-0.42
3	0.14	-0.99

"bank at position 6" ≠ bank at position 2"
even if the word is identical

Why Self-attention? (Why Attention matter?)

Embeddings alone don't understand order.

"I deposited money in the bank"

"I sat on the river bank"

The word **bank** has two meanings:

-  financial institution
-  river edge

Step1: Token embeddings

Let's simplify embeddings to 3D vectors:

bank = [0.5, 0.5, 0.5]
money = [0.9, 0.1, 0.2]
river = [0.2, 0.9, 0.3]
deposited = [0.8, 0.2, 0.4]
sat = [0.3, 0.7, 0.6]

Step2: Add positional encoding

Sentence 1:

I deposited money in the bank

Sentence 2:

I sat on the river bank

Even after positional encoding, **bank is still ambiguous**.
So we need **attention**.

Full attention matrix view

For every word, transformer builds a grid like this:

	I	deposited	money	river	bank
I	0	0.1	0.1	0.0	0.1
deposited	0	0.1	0.3	0.0	0.2
money	0	0.2	0.2	0.0	0.1
river	0	0.0	0.0	0.2	0.4
bank	0	0.3	0.4	0.5	0.1

This is usually visualized as a **heatmap** where:

- bright = strong attention
- dark = weak attention

Multi-head Attention

- grammar (who did what)
- Semantic meaning (financial vs river bank)
- relationships (verb-object links)

Concatenation of Multiple Attention Heads

Head 1 → [0.8, 0.1] (meaning)
Head 2 → [0.3, 0.9] (grammar)
Head 3 → [0.6, 0.4] (context)

We combine them:

BANK_final = concat(H1, H2, H3)

BANK

Step 3: Self-attention idea

We compute:

- Query (Q): what I'm looking for
- Key (K): what I represent
- Value (V): what I contain

For simplicity, assume:

Q = K = V = embeddings

Case A: "bank" in financial context

Sentence:

I deposited money in the bank

We compute similarity of **bank with other words**.

Dot products (simplified):

bank · money = HIGH (0.5·0.9 + ...)

bank · deposited = HIGH

bank · river = LOW

bank · sat = LOW

Softmax result (attention weights):

money → 0.45

deposited → 0.35

others → 0.20

New representation of "bank":

bank_finance =

0.45·money + 0.35·deposited + 0.20·others

Case B: "bank" in river context

Sentence:

I sat on the river bank

Now attention changes:

bank · river = HIGH

bank · sat = MEDIUM

bank · money = LOW

Softmax result (attention weights):

river → 0.50

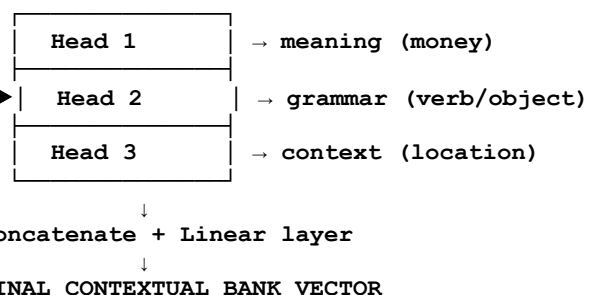
sat → 0.30

others → 0.20

New representation:

bank_river =

0.50·river + 0.30·sat + 0.20·others



Full transformer forward pass walk through

We'll use a simple sentence:

"I deposited money in the bank"

And we'll pretend the model is trying to predict the **next word after "bank"**.

OVERVIEW:

Tokens → Embeddings (meaning) → Positional Encoding (order) → Multi-Head Self-Attention (relationships) → Feed Forward Network (feature refinement) → Final Linear Layer (map to vocabulary) → Softmax (probabilities) → Prediction / NEXT WORD

STEP 1: Tokenization

We convert text into token IDs:

"I deposited money in the bank"
→ [I, deposited, money, in, the, bank]
→ [10, 532, 98, 44, 21, 77]

STEP 2: Token Embeddings

Each token becomes a vector (say 4D for simplicity):

I → [0.2, 0.1, 0.7, 0.0]
deposited → [0.6, 0.2, 0.1, 0.9]
money → [0.9, 0.1, 0.3, 0.4]
in → [0.3, 0.8, 0.2, 0.1]
the → [0.5, 0.4, 0.6, 0.2]
bank → [0.7, 0.3, 0.8, 0.5]

Now we have a matrix: **X = (6 tokens × 4 dims)**

STEP 3: Positional Encoding (adds order)

We add position vectors:

P0 = [0.0, 0.1, 0.0, 0.1]
P1 = [0.1, 0.0, 0.1, 0.0]
P2 = ...

Example:

bank = [0.7, 0.3, 0.8, 0.5]
P5 = [0.1, 0.0, 0.1, 0.0]

→ bank' = [0.8, 0.3, 0.9, 0.5]

Final input: **X' = embedding + position**

STEP 4: Self-Attention

We compute for each token:

Q (Query), K (Key), V (Value)

Q = XW_q
K = XW_k
V = XW_v

Focus on ONE token: "bank"

We compute how "bank" attends to all tokens:

Step 4.1: similarity scores

bank · I = 0.2
bank · money = 0.9 ← strong
bank · deposited = 0.85 ← strong
bank · in = 0.3
bank · the = 0.4
bank · bank = 0.6

Step 4.2: softmax (attention weights)

I → 0.05
deposited → 0.30
money → 0.35
in → 0.05
the → 0.05
bank → 0.20

Step 4.3: weighted sum (new representation)

bank_context = 0.05 · I + 0.30 · deposited + 0.35 · money + 0.20 · bank + ...

👉 Now "bank" becomes **financial-context-aware**

STEP 5: Multi-head attention (parallel views)

concat(head1, head2, head3) → linear transform

STEP 7: Final Linear Layer (vocabulary projection)

We now predict next word:

logits = hidden_state(bank) · W_vocab

STEP 6: Feed Forward Network (MLP per token)

Each token independently passes through:

FFN(x) = W₂(ReLU(W₁x))

STEP 8: Softmax

Convert logits into probabilities:

P("loan") = 0.30
P("account") = 0.25
P("river") = 0.01

Model outputs: "I deposited money in the bank → loan"

Causal Self Attention (self-attention + masking constraint)

Self-attention with Q, K, and V tells the model **how tokens interact**.

Causal attention tells the model **which tokens are allowed to interact**.

Regular self-attention:

In self-attention, every token can attend to every other token.

For a sequence:

"I love deep learning"

each word can look at all words, including future ones.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Nothing here prevents token 1 from seeing token 4.

So during training:

- "I" can see "learning"
- "love" can see "deep"
- etc.

This is fine for tasks where the whole sentence is available:

- translation encoder
- BERT
- sentiment classification
-

Why causal attention is needed?

For text generation, the model must predict the **next token** without cheating.

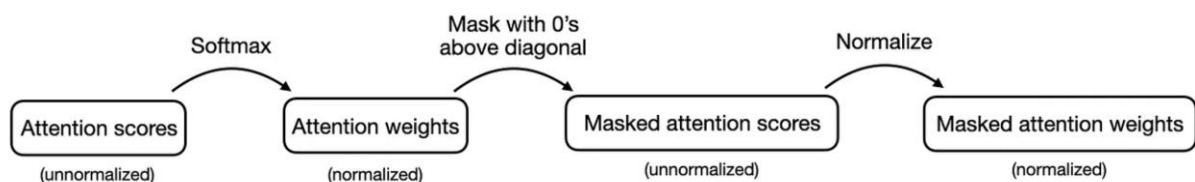
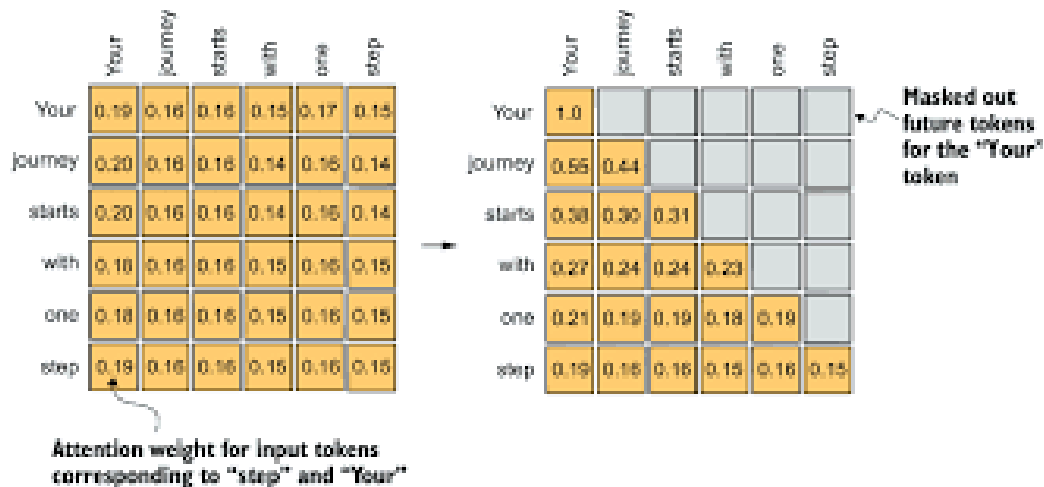
What causal attention does?

Causal attention adds a **mask** to self-attention.

Token at position i can only attend to positions $\leq i$.

Token	Can attend to
The	The
Cat	The, cat
Sat	The, cat, sat
on	The, cat, sat, on

Future tokens are masked with $-\infty$ before softmax.



Alternatively, there is a more efficient way to achieve the same results.



Implement Self Attention

```
import torch.nn as nn
```

```
class SelfAttentionV1(nn.Module):
```

```
    def __init__(self, d_in, d_out, qkv_bias=False):
```

```
        super().__init__()
```

```
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
```

```
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
```

```
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
```

```
    def forward(self, x):
```

```
        keys = x @ self.W_key(x)
```

```
        queries = x @ self.W_query(x)
```

```
        values = x @ self.W_value(x)
```

```
        attn_scores = queries @ keys.T # omega
```

```
        attn_weights = torch.softmax(
            attn_scores / keys.shape[-1] ** 0.5, dim=-1
        )
```

```
        context_vec = attn_weights @ values
```

```
        return context_vec
```

More Efficient ways

Attention Scores --> Upper Triangular Infinity

Mask --> Softmax

```
torch.triu(torch.ones(ctx_length, ctx_length))
```

```
(([1., 1., 1., 1., 1., 1.],
  [0., 1., 1., 1., 1., 1.],
  [0., 0., 1., 1., 1., 1.],
  [0., 0., 0., 1., 1., 1.],
  [0., 0., 0., 0., 1., 1.],
  [0., 0., 0., 0., 0., 1.]])
```

```
mask = torch.triu(torch.ones(ctx_length,
                               ctx_length), diagonal=1)
masked = attn_scores.masked_fill(mask.bool(),
                                  -torch.inf)
print(masked)
```



```
torch.manual_seed(123)
```

```
sa_v1 = SelfAttentionV1(d_in, d_out)
```

```
print(sa_v1(inputs))
```

Apply softmax

```
attn_weights = torch.softmax(
    masked / keys.shape[-1] ** 0.5, dim=1
)
```

```
class CausalAttention(nn.Module):
```

```
    def __init__(self, d_in, d_out, ctx_length, dropout, qkv_bias=False):
```

```
        super().__init__()
```

```
        self.d_out = d_out
```

```
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
```

```
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
```

```
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
```

```
        self.dropout = nn.Dropout(dropout)
```

```
        self.register_buffer('mask', torch.triu(torch.ones(ctx_length, ctx_length), diagonal=1))
```

```
    def forward(self, x):
```

```
        b, num_tokens, d_in = x.shape # new batch dimension b
```

```
        keys = self.W_key(x)
```

```
        queries = self.W_query(x)
```

```
        values = self.W_value(x)
```

```
        attn_scores = queries @ keys.transpose(-2, -1) # omega
```

```
        attn_scores.masked_fill(
            self.mask.bool()[:num_tokens, :num_tokens], -torch.inf
        )
```

```
        attn_weights = torch.softmax(attn_scores / keys.shape[-1] ** 0.5, dim=1)
```

```
        attn_weights = self.dropout(attn_weights)
```

```
        context_vec = attn_weights @ values
```

```
        return context_vec
```

Why dropout is applied after attention scores?

The attention mechanism computes: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

The important part is: $A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$

where A is the **attention weight matrix**.

Dropout is typically applied like this: $\text{Dropout}(A)V$

Why dropout is needed here

Without dropout:

- the model may always focus on the same tokens,
- attention heads can become overly confident,
- the model overfits training data,
- some heads become redundant.

Applying dropout forces the model to distribute attention more robustly.

Intuition

Think of attention like students copying answers.

If one student always copies from the same topper, they fail when that topper is absent.

Dropout randomly blocks some “copy connections” during training so the student learns from multiple sources.

Example:

"The cat sat on the mat"

After softmax, attention weights may become:

Token	Attention weight
The	0.05
cat	0.70
sat	0.10
on	0.05
the	0.05
mat	0.05

Here the model heavily depends on "cat".

Without dropout The output becomes:

$$0.70 \cdot V_{\text{cat}} + 0.10 \cdot V_{\text{sat}} + \dots$$

The model learns:

“Whenever predicting sat, just look at cat.”

➔ This can overfit.

With dropout (say p=0.5)

During training, some attention weights are randomly zeroed.

Token	Original	After dropout
The	0.05	0
cat	0.70	0
sat	0.10	0.20
on	0.05	0.10
the	0.05	0
mat	0.05	0.10

(remaining weights are rescaled)

Now "sat" cannot rely only on "cat".

Why dropout is applied AFTER softmax?

Before softmax, scores are raw logits: QK^T

These are unnormalized and unstable.

After softmax:

- values become probabilities,
- easier to interpret,
- dropout removes actual attention links.

So the model randomly removes **connections between tokens**.

What exactly is being dropped?

Not tokens.

Not values V .

The model drops **attention probabilities**.

So certain token-to-token interactions disappear temporarily during training.

In PyTorch

```
attn = softmax(scores, dim=-1)
attn = dropout(attn)
output = attn @ V
```

Multi-Head Attention

Model	Parameters	Transformer Layers	Attention Heads
GPT-3 (175B version)	175 billion	96	96 heads
GPT-4	Not officially disclosed	Not officially disclosed	Unknown / not publicly released
DeepSeek-V3	671B total (37B active)	128 Multi-head Latent Attention (MLA)	128 heads

For the largest GPT-3 model (175B), OpenAI publicly reported:

- **96 transformer layers**
- **96 attention heads**
- hidden size $d_{model} = 12,288$
- head dimension = 128

So: $96 \times 128 = 12288$

which matches the model embedding dimension.

GPT-3 architecture summary

Component	Value
Parameters	175B
Layers	96
Attention heads	96
Head dimension	128
Hidden size	12,288
Context window	2048 tokens

GPT-3 uses standard: $\text{softmax}(QK^T)V$
But DeepSeek-V2/V3 introduced: **MLA**

MLA — Multi-head Latent Attention:

Instead of storing full Keys/Values for every head, DeepSeek compresses them into a smaller latent representation.

this reduces:

- KV cache memory,
- inference cost,
- bandwidth usage.

So even though DeepSeek has **128 heads**, the memory cost is much lower than traditional MHA.

```
class MultiHeadAttentionWrapper(nn.Module):
    def __init__(self, d_in, d_out, ctx_length, dropout, num_heads, qkv_bias=False):
        super().__init__()
        self.num_heads = nn.ModuleList([
            CausalAttention(d_in, d_out, ctx_length, dropout, qkv_bias=qkv_bias)
            for _ in range(num_heads)
        ])

    def forward(self, x):
        return torch.cat([head(x) for head in self.heads], dim=-1)
```

Multi-Head Attention with Weight Split — Intuition

Instead of using one large attention operation, the model splits the representation into multiple smaller parts (heads), and each head learns different attention patterns using different weights.

Step 1 — Single-head attention problem

Suppose a word embedding has dimension: $d_{model} = 512$

If we use only one attention mechanism:

- one Query matrix W_Q
- one Key matrix W_K
- one Value matrix W_V

then the model learns only **one type of relationship**.

Step 2 — Split into multiple heads

Instead of one big attention:

Transformer splits the 512-dimensional vector into smaller chunks.

Example:

$h = 8$ heads

Then each head gets: $d_k = \frac{512}{8} = 64$

So:

Head	Vector size
Head 1	64
Head 2	64
Head 3	64
...	
Head 8	64

ss

What “weight split” means

Each head has its **own learned weight matrices**:

$$W_Q^i, W_K^i, W_V^i$$

for head i .

So the model learns different projections for each head.

Visualization

Instead of:

512-dim vector

↓

One attention block

we do:

512-dim vector

↓ split

[64][64][64][64][64][64][64][64]

↓

8 independent attention heads

Each head learns something different.

Layer Normalization in Transformers

Without it:

- values can grow too large,
- gradients can explode or vanish,
- training becomes unstable,
- deep Transformers become hard to optimize.

For a token vector:

[2.1, -0.5, 8.7, 15.2, ...]

LayerNorm transforms it into something like:

[-0.2, -1.1, 0.5, 1.3, ...]

So the values stay centered and scaled.

What LayerNorm does mathematically

Given a token representation: $x = [x_1, x_2, \dots, x_d]$

LayerNorm computes:

$$\text{Mean } \mu = \frac{1}{d} \sum x_i$$

$$\text{Variance } \sigma^2 = \frac{1}{d} \sum (x_i - \mu)^2$$

$$\text{Normalize } \hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Then apply learnable scale and shift:

$$y_i = \gamma \hat{x}_i + \beta$$

where:

- γ = learnable scale
- β = learnable shift

Why not BatchNorm?

BatchNorm	LayerNorm
depends on batch statistics	independent of batch
bad for variable sequence lengths	works well for NLP
problematic for autoregressive generation	perfect for token-wise processing

Transformers process sequences token-by-token, so LayerNorm is more suitable.

Analogy

Imagine students taking exam

Without normalization:

- one exam may be out of 10,
- another out of 1000,
- another out of 50.

Comparisons become unstable.

LayerNorm rescales everything consistently.

Important intuition

LayerNorm does NOT add knowledge.

It helps the network maintain:

- stable numerical behavior,
- smooth optimization,
- trainability at large scale.

It is mainly an optimization/stability tool.

Ex: $x = [1.1, 0.8, 2.3, 4.4]$

$$\text{Mean } \mu = \frac{1.1 + 0.8 + 2.3 + 4.4}{4} = \frac{8.6}{4} = 2.15$$

$$\begin{aligned} \text{Variance } \sigma^2 &= \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2 \\ &= \frac{1}{4} [(x_1 - \mu)^2 + (x_2 - \mu)^2 + (x_3 - \mu)^2 + (x_4 - \mu)^2] = 2.0025 \end{aligned}$$

$$\text{Standard Deviation } \sigma = \sqrt{\sigma^2} = \sqrt{2.0025} \approx 1.415$$

$$\begin{aligned} \text{Normalized} &= \left[\frac{x_1 - \mu}{\sigma}, \frac{x_2 - \mu}{\sigma}, \frac{x_3 - \mu}{\sigma}, \frac{x_4 - \mu}{\sigma} \right] \\ &= \left[\frac{1.1 - 2.15}{1.415}, \frac{0.8 - 2.15}{1.415}, \frac{2.3 - 2.15}{1.415}, \frac{4.4 - 2.15}{1.415} \right] \\ &= [-0.742, -0.954, 0.106, 1.590] \end{aligned}$$

Note about real Transformers

Actual LayerNorm uses: $\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$

where:

- ϵ is a tiny number like 10^{-5}
- added for numerical stability.

Then Transformers apply learnable parameters:

$$y_i = \gamma \hat{x}_i + \beta$$

Coding Layer Normalization:

```
class LayerNorm(nn.Module):
    def __init__(self, emb_dim):
        super().__init__()
        self.eps = 1e-5
        self.scale = nn.Parameter(torch.ones(emb_dim))
        self.shift = nn.Parameter(torch.zeros(emb_dim))

    def forward(self, x):
        mean = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)
        norm_x = (x - mean) / torch.sqrt(var + self.eps)
        return self.scale * norm_x + self.shift
```

GELU Activation Function (Gaussian Error Linear Unit)

GELU decides:

“How much of the input should pass through?”

But unlike ReLU, it does this **smoothly and probabilistically**.

$$\text{GELU}(x) = x \cdot \Phi(x)$$

where: $\Phi(x)$: is the cumulative distribution function (CDF) of a standard Gaussian distribution.

$$\text{GELU}(x) = 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right)$$

Intuition:

small negative values → partially suppressed

large negative values → near zero

positive values → mostly preserved

Unlike ReLU, it does NOT sharply cut values at zero.

Why "Gaussian"?

Because GELU uses Gaussian probability: $\Phi(x)$

Interpretation: $P(X \leq x)$

You can think of GELU as: “Randomly keep neurons depending on their importance.”

- Large positive values → highly preserved.
- Small negative values: → softly reduced.
- Very negative values: → nearly removed.

Example Calculation:

Suppose: $x = 1$

Approximate: $\Phi(1) \approx 0.841$

Then: $\text{GELU}(1) = 1 \times 0.841 = 0.841$

For: $x = -1 \rightarrow \Phi(-1) \approx 0.159$

Then: $\text{GELU}(-1) = -1 \times 0.159 = -0.159$

Notice:

- negative values are not fully zeroed,
- unlike ReLU.

Why Feed Forward Neural Network? (Why attention alone is insufficient)

Attention computes relationships:

$$\text{softmax}(QK^T)V$$

This tells the model:

- which words matter,
- how strongly they interact.

But attention is mostly a **weighted averaging mechanism**.

Weighted averaging alone cannot learn very complex transformations.

Example:

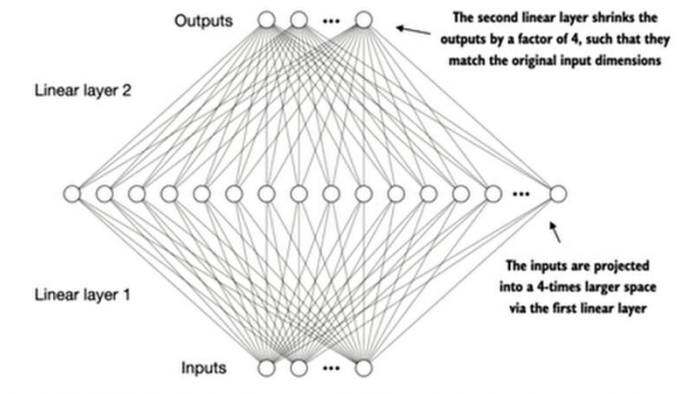
"The movie was surprisingly good"

Attention may learn:

- "good" relates to "movie"
- "surprisingly" modifies "good"

But attention itself cannot strongly transform:

- sentiment,
- abstraction,
- semantic composition.



Main purpose of FFN

1. Adds nonlinearity
2. Feature transformation
3. Expands representational capacity

1. Adds nonlinearity:

- Without FFN: Transformer becomes mostly linear operations.
- Stacking linear layers without nonlinearity collapses into another linear transformation.
- GELU introduces nonlinear learning capability.

Why nonlinearity matters? Language is highly nonlinear.

Example: "not bad" ≠ negative → Meaning changes contextually.

2. Feature transformation

After attention:

[cat-related info,
grammar info,
position info]

FFN converts it into:

- higher-level semantic features,
- abstract concepts,
- prediction-friendly representations.

3. Expands representational capacity

Transformer FFNs usually expand dimensions:

Example GPT/BERT style:

$$768 \rightarrow (4 * 768) \ 3072 \rightarrow 768$$

Example:

Input embedding: 768 dimensions

First linear layer: $768 \rightarrow 3072$

Now the network has more room to:

- combine features,
- detect patterns,
- build abstractions.

Second linear layer compresses back:

$$3072 \rightarrow 768$$

Why Shortcut (Residual) Connections are Needed in Transformers

Residual Connections or Skip Connections — are critical for training deep Transformers.

They help:

- preserve information,
- stabilize gradients,
- prevent degradation in deep networks,
- make training much easier.

What is a shortcut connection?

Instead of using only: **Output** = $F(x)$

Transformer uses: **Output** = $x + F(x)$

where:

- x = original input,
- $F(x)$ = transformed output (attention or FFN).

So the input is directly added back.

Why is this needed?

Deep networks have a major problem:

Problem: information degradation

As layers increase:

Layer 1 → good signal
Layer 10 → weaker
Layer 50 → distorted
Layer 100 → almost lost

Important information can vanish.

Residual connections preserve the original signal.

1. Prevent vanishing gradients

During backpropagation gradients must travel through many layers.

Without residuals:

gradient × small numbers × small numbers × ...

Eventually gradients become tiny ≈ 0

→ Training stops learning.

With shortcut connection

Gradient gets a direct path: $\frac{\partial}{\partial x} (x + F(x)) = 1 + F'(x)$

Notice the important part:

2. Preserve original information:

Attention/FFN may distort useful features.

Residual keeps the original representation available.

Example:

Output = Original Meaning + Learned Modification

instead of completely overwriting it.

Example

Suppose token embedding:

x = [cat, animal, pet]

Attention learns contextual info:

$F(x)$ = [sits on mat]

Residual output:

$x + F(x)$ = [cat, animal, pet, sits on mat]

Original semantics are preserved.

3. Makes deep Transformers trainable

Without residuals:

- deep models fail,
- optimization becomes unstable.